

МИНОБРНАУКИ РОССИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ «ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ»

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Лабораторная работа №4 по дисциплине «Архитектура информационных
систем»

Обучающийся

М.А.Ячный, 4 курс, группа 9.1

Руководитель

Е.В. Попова, старший преподаватель

Системный паттерн «Текущие обновления» («Successive updates»)

Название: Текущие обновления (Successive updates)

Тип: Системный паттерн

Назначение: Обеспечение клиенту возможности постоянного получения обновлений от сервера. Такие обновления обычно отражают изменения данных сервера, впервые появившиеся или обновленные ресурсы либо изменения в состоянии бизнес-модели.

Другие названия: Long Polling

Мотивация (решаемые проблемы):

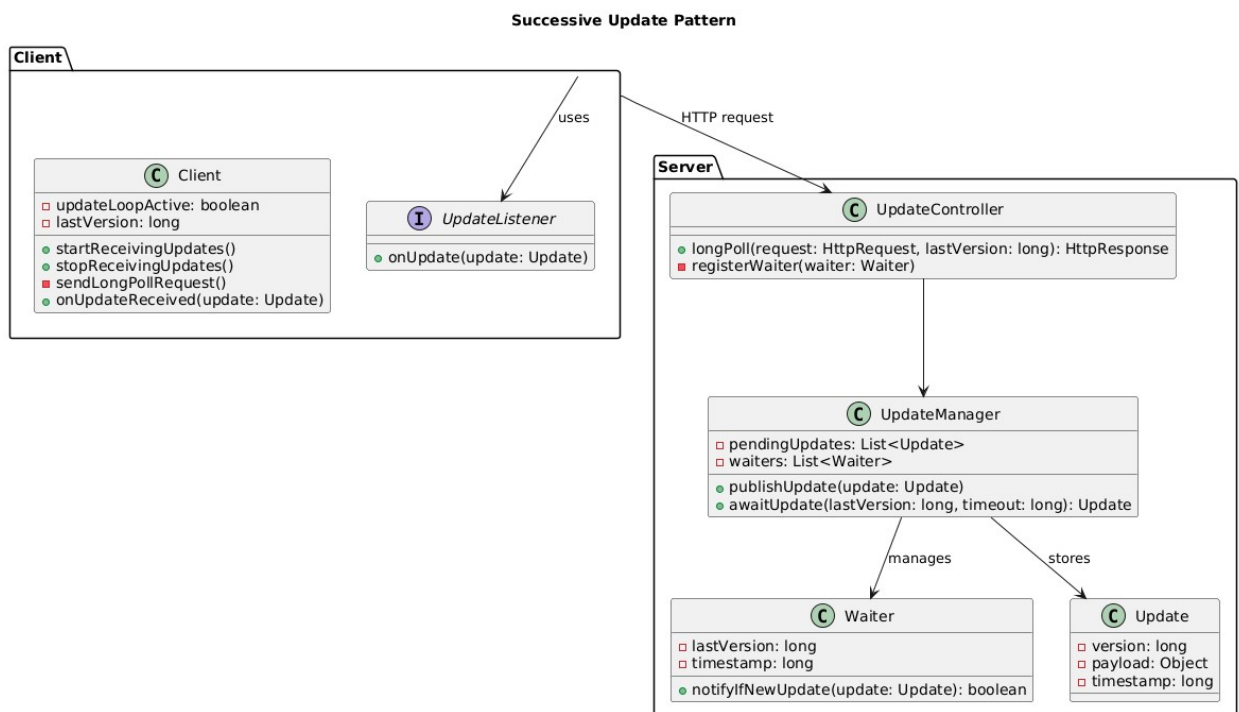
1. Невозможность сервера инициировать соединение – в классической клиент-серверной архитектуре на базе HTTP сервер не может самостоятельно отправить данные клиенту (например, из-за NAT, фаерволов или stateless-природы протокола). Successive updates позволяет серверу "дождаться" момента, когда данные появятся, и отправить их в уже открытом соединении.
2. Избыточная нагрузка от короткого опроса – при коротком опросе клиент отправляет запросы каждые N секунд, даже если данных нет. Это создаёт холостую нагрузку на сеть, сервер и базу данных. Successive updates отправляет ответ только при наличии данных, сокращая количество пустых запросов.
3. Минимизация задержек доставки – в отличие от короткого опроса, где задержка составляет несколько секунд, Long Polling доставляет данные практически мгновенно (сразу после появления).

Условия, в которых целесообразно применять паттерн:

1. Невысокая частота событий – когда обновления происходят не постоянно, а эпизодически. При очень высокой частоте (например, тысячи событий в секунду) WebSockets эффективнее.

2. Требуется совместимость с HTTP/1.1 – WebSockets, решающие похожую задачу, требуют обновления протокола.
3. Простота реализации на клиенте – не нужны специальные API (в отличие от WebSocket), достаточно стандартного XMLHttpRequest или fetch.
4. Односторонний поток данных – когда клиенту нужно только получать обновления от сервера, а не отправлять их непрерывно в обе стороны.

Структура паттерна (диаграмма классов):



1. Клиентский слой – класс Client управляет жизненным циклом получения обновлений: запускает цикл длинных запросов (startReceivingUpdates()), хранит последнюю полученную версию (lastVersion) и обрабатывает входящие обновления через onUpdateReceived(). Интерфейс UpdateListener обеспечивает гибкую обработку обновлений в UI-компонентах.
2. Инициация запроса – Client отправляет HTTP-запрос к серверному UpdateController, передавая параметр lastVersion. Это позволяет серверу

определить, есть ли у клиента устаревшие данные, и отдать обновление немедленно, минуя постановку в очередь ожидания.

3. Управление ожиданием – UpdateController делегирует логику ожидания UpdateManager, который ведёт два ключевых хранилища: список опубликованных обновлений (pendingUpdates) и очередь ожидающих клиентов (waiters). Каждый ожидающий представлен объектом Waiter, содержащим версию клиента и временную метку для контроля таймаутов.
4. Публикация и уведомление – при появлении нового Update (вызов publishUpdate()) UpdateManager проходит по списку waiters и через метод notifyIfNewUpdate() проверяет, какие из ожидающих клиентов могут получить это обновление. После уведомления Waiter удаляется из очереди, а клиенту возвращается HTTP-ответ с данными.

Объекты и паттерны, используемые в данном паттерне:

Request Manager	Управляет жизненным циклом запросов: отправляет запрос, обрабатывает ответ, инициирует следующий. Реализует повторные попытки при ошибках.
Event Bus	На сервере – шина событий, на которую публикуются изменения данных. Клиенты, ожидающие обновлений, подписываются на неё.
Delayed Response / Deferred Result	Механизм, позволяющий серверу не возвращать ответ немедленно, а отложить его до наступления события или таймаута.
Timeout Manager	Контролирует максимальное время ожидания ответа на сервере и на клиенте, предотвращая зависание соединений.

Correlation ID / Last Event ID	Идентификатор последнего полученного события, передаваемый в каждом запросе, чтобы сервер мог вернуть только новые данные.
Observer	Клиент выступает наблюдателем, подписанным на серверные события через механизм long polling.
Retry with Backoff	Стратегия повторных запросов при сбоях

Результаты работы паттерна:

Положительные результаты:

1. Мгновенная доставка данных – клиент получает обновления сразу после их появления (с минимальной задержкой, определяемой временем обработки).
2. Снижение нагрузки по сравнению с простым опросом – отсутствуют пустые запросы без данных.
3. Высокая совместимость – работает через стандартный HTTP, не требует специальных протоколов или специфических настроек сетевого оборудования.
4. Прозрачность для инфраструктуры – балансировщики нагрузки и прокси корректно обрабатывают длительные соединения в пределах установленных таймаутов.
5. Простота клиентской реализации – клиенту достаточно уметь выполнять HTTP-запросы и обрабатывать ответы.

Отрицательные результаты и ограничения:

1. Расход ресурсов сервера – каждое удерживаемое соединение потребляет память и файловые дескрипторы.

2. Невозможность гарантировать порядок при сетевых разрывах – при обрыве соединения клиент может пропустить события, если они не хранятся на сервере для повторной отправки.
3. Оверхед на установку соединений – каждый цикл требует нового TCP-соединения (в HTTP/1.1) или повторного использования с задержкой.

Рекомендации по применению:

1. Устанавливать разумные таймауты, обычно 20–60 секунд
2. Реализовать механизм id последнего события. Клиент передаёт идентификатор последнего обработанного события. Сервер возвращает только события, произошедшие после этого идентификатора, что обеспечивает устойчивость к разрывам соединения.
3. Правильно ошибки с умом: При сетевых ошибках клиент должен повторять запрос с экспоненциальной задержкой.
4. Хранить события для повторной доставки, использовать Kafka/Redis Streams, чтобы клиент мог получить пропущенные события после переподключения.

Пример кода:

Клиентская часть

```
public class SuccessiveUpdateClient {

    private final HttpClient httpClient;
    private final String serverUrl;
    private String lastEventId = null;
    private volatile boolean running = true;

    public SuccessiveUpdateClient(String serverUrl) {
        this.serverUrl = serverUrl;
        this.httpClient = HttpClient.newHttpClient();
    }
}
```

```

public void start() {
    new Thread(this::pollLoop).start();
}

public void stop() {
    running = false;
}

private void pollLoop() {
    while (running) {
        try {
            String url = serverUrl + (lastEventId != null
                ? "?lastEventId=" + lastEventId
                : "");

            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .timeout(Duration.ofSeconds(30))
                .GET()
                .build();

            HttpResponse<String> response = httpClient.send(
                request,
                HttpResponse.BodyHandlers.ofString());

            if (response.statusCode() == 200) {
                String responseBody = response.body();
                processUpdate(responseBody);
            } else if (response.statusCode() == 204) {
                Thread.sleep(100);
            } else {
                handleError(response.statusCode());
            }
        } catch (HttpTimeoutException e) {
            continue;
        } catch (Exception e) {
            handleException(e);
        }
    }
}

```

```

private void processUpdate(String data) {
    lastEventId = extractEventId(data);
    System.out.println("Получено обновление: " + data);
    onUpdateReceived(data);
}

private String extractEventId(String json) {
    return json.replaceAll(".*\"eventId\":\\"([^\"]+)\\".*",
"$1");
}

private void onUpdateReceived(String data) {
    //[какая-то обработка]
}

private void handleError(int statusCode) {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException ignored) {}
}

private void handleException(Exception e) {
    e.printStackTrace();
    try {
        Thread.sleep(1000);
    } catch (InterruptedException ignored) {}
}
}

```

Серверная часть

```

public class UpdatesController {

    private final EventBus eventBus;

    public void getUpdates(@RequestParam(required = false)
String lastEventId,
                        HttpServletRequest request,
                        AsyncContext asyncContext) {
        EventListener listener = (event) -> {
            if (event.getId().compareTo(lastEventId) > 0) {

```



```

asyncContext.getResponse().getWriter().write(event.toJson());
        asyncContext.complete();
    }
};

eventBus.subscribe(listener);

asyncContext.addListener(new AsyncListener() {
    @Override
    public void onTimeout(AsyncEvent event) {
        eventBus.unsubscribe(listener);
        asyncContext.getResponse().setStatus(204);
        asyncContext.complete();
    }

    @Override
    public void onComplete(AsyncEvent event) {
        eventBus.unsubscribe(listener);
    }

});
}
}

```

Пояснение к примеру:

- Клиент запускает бесконечный цикл, в котором последовательно выполняет HTTP GET-запросы к серверу.
- Каждый запрос содержит идентификатор `lastEventId` последнего полученного события. Это позволяет серверу вернуть только новые данные.
- Сервер принимает запрос, но не отвечает немедленно. Он регистрирует слушателя на шине событий и удерживает соединение (асинхронный контекст).
- При появлении нового события (например, изменение бизнес-модели) сервер отправляет ответ клиенту и завершает соединение.

- Клиент, получив ответ, обрабатывает данные, сохраняет новый lastEventId и сразу же выполняет следующий запрос, замыкая цикл.
- Если событий нет в течение таймаута, сервер возвращает ответ 204 No Content, клиент делает небольшую паузу и продолжает опрос.

Примеры использования:

- Системы совместной работы (Jira, Google docs): уведомления об обновлениях на досках
- Мессенджеры: уведомления о новых сообщениях
- Мониторинг и алерты: мониторинг метрик (uptime, нагрузка на процессор и т.п.) и уведомления о превышении значений

Родственные паттерны:

1. Short polling – Более простая, но менее эффективная альтернатива. Клиент опрашивает сервер через фиксированные интервалы независимо от наличия данных. Successive Update – эволюция этого подхода с уменьшением задержки и нагрузки.
2. Websocket – Более современная альтернатива для двунаправленной связи с постоянным соединением. Successive Update часто используется как fallback, когда WebSockets недоступны.
3. Producer-Subscriber – Подписчики (subscribers) подписываются на сообщения, рассылаемые издателями (producers).

Структурный паттерн «Декоратор» («Decorator»)

Название: Декоратор (Decorator)

Тип: Структурный паттерн

Назначение: Динамически добавлять объекту новые обязанности (поведение) без изменения его исходного кода и без использования наследования.

Другие названия: Wrapper

Мотивация (решаемые проблемы):

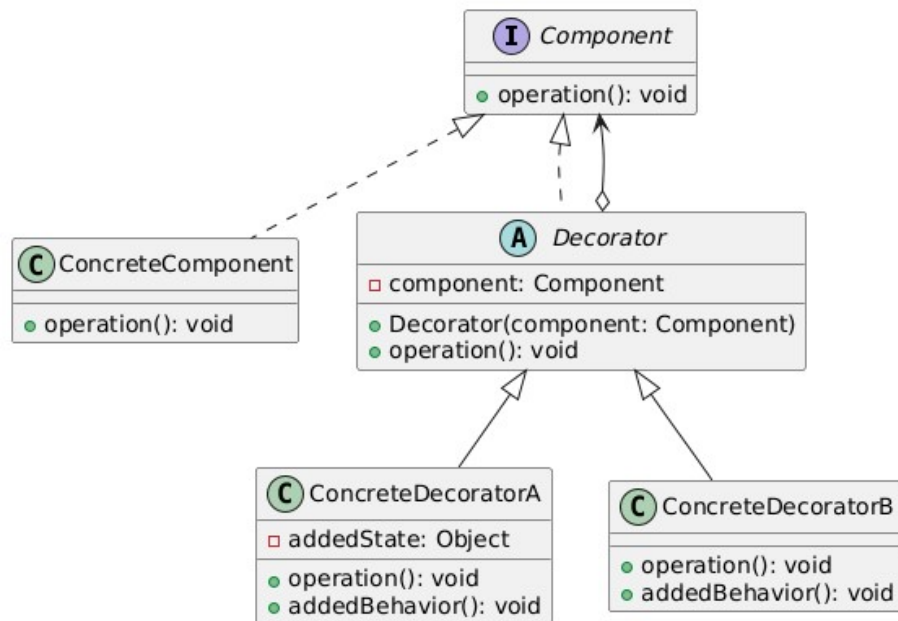
1. Взрыв количества классов в иерархии – для каждой комбинации функциональности пришлось бы создавать отдельный подкласс, что приводит к экспоненциальному росту числа классов.
2. Статичность наследования – функциональность, добавленная через наследование, фиксируется на этапе компиляции и не может быть изменена во время выполнения.
3. Нарушение принципа открытости/закрытости – добавление новой функциональности требует изменения существующего кода или создания новых подклассов, что может быть нежелательно для уже протестированных и стабильных классов.
4. Перегрузка интерфейса – попытка предусмотреть все возможные варианты поведения в базовом классе приводит к раздуванию интерфейса и усложнению его поддержки.

Условия, в которых целесообразно применять паттерн:

1. Необходимо добавлять обязанности объектам динамически, во время выполнения – когда обновления происходят не постоянно, а эпизодически. При очень высокой частоте (например, тысячи событий в секунду) WebSockets эффективнее.

2. Существует несколько независимых расширений функциональности, которые могут комбинироваться в различных сочетаниях.
3. Недопустимо или затруднительно использование наследования – например, когда класс объявлен как `final`, либо когда наследование приведёт к неоправданному разрастанию иерархии.

Структура паттерна (диаграмма классов):



1. Базовый интерфейс – **Component** определяет общий контракт (`operation()`), которому следуют как конкретные реализации, так и декораторы. Это позволяет клиентам работать с обернутыми объектами единообразно.
2. Исходный объект – **ConcreteComponent** представляет собой базовую реализацию компонента, которую необходимо расширить дополнительной функциональностью. Он не знает о существовании декораторов.
3. Абстрактный декоратор – **Decorator** содержит ссылку (`component`) на обернутый объект того же типа **Component** и делегирует ему вызов `operation()`. Абстрактный класс обеспечивает единую структуру для всех конкретных декораторов.

4. Конкретные декораторы – ConcreteDecoratorA и ConcreteDecoratorB расширяют поведение базового компонента. Они переопределяют operation(), добавляя собственную логику (например, дополнительные вычисления, логирование, валидацию) до или после вызова operation() у обёрнутого объекта.

Объекты и паттерны, используемые в данном паттерне:

Component (интерфейс)	Определяет единый контракт, который позволяет декораторам быть взаимозаменяемыми с конкретными компонентами.
ConcreteComponent	Исходный объект, к которому добавляется функциональность. Содержит базовую реализацию.
Decorator (абстрактный класс)	Хранит ссылку на обёрнутый компонент. Обеспечивает единообразное делегирование вызовов.
ConcreteDecorator	Реализует конкретное расширение функциональности. Может добавлять состояние и поведение.

Результаты работы паттерна:

Положительные результаты:

1. Гибкость добавления функциональности – обязанности могут добавляться и удаляться во время выполнения, в отличие от статического наследования.

2. Избежание взрыва иерархии – вместо $N \times M$ подклассов для N базовых типов и M расширений достаточно N конкретных компонентов и M декораторов.
3. Соблюдение принципа открытости/закрытости – классы остаются закрытыми для модификации, но открытыми для расширения через декорирование.
4. Комбинируемость – декораторы могут накладываться друг на друга в любом порядке, создавая разнообразные комбинации поведения.

Отрицательные результаты и ограничения:

1. Сложность отладки – стек вызовов становится менее очевидным, так как включает множество вложенных декораторов.
2. Чувствительность к порядку – результат зависит от порядка наложения декораторов, что может приводить к неожиданному поведению при неправильной конфигурации.
3. Ограниченная поддержка идентичности – объект, обёрнутый в декоратор, перестаёт быть идентичным исходному объекту (`equals()`, `hashCode()` могут работать некорректно).

Рекомендации по применению:

1. Обеспечить единый интерфейс – все декораторы и конкретные компоненты должны реализовывать общий интерфейс `Component`. Это гарантирует прозрачность использования.
2. Делегировать вызовы базовому компоненту – абстрактный декоратор должен содержать ссылку на `Component` и делегировать все вызовы. Конкретные декораторы вызывают `super` для сохранения цепочки.
3. Проектировать декораторы как независимые слои – каждый декоратор должен добавлять ровно одну чётко определённую ответственность (принцип единой ответственности).

4. Документировать порядок наложения – если порядок декораторов влияет на результат, это должно быть явно задокументировано или контролироваться через фабричные методы.

Пример кода:

Общий интерфейс:

```
public interface Notifier {  
    void send(String message);  
}
```

Конкретный компонент:

```
// Базовая реализация - отправка через email  
public class EmailNotifier implements Notifier {  
    private final String email;  
  
    public EmailNotifier(String email) {  
        this.email = email;  
    }  
  
    @Override  
    public void send(String message) {  
        // Упрощённо: вывод в консоль вместо реальной отправки  
        System.out.println("Отправка email на " + email + ": " +  
message);  
    }  
}
```

Абстрактный декоратор:

```
// Абстрактный декоратор, оборачивающий любой Notifier  
public abstract class NotifierDecorator implements Notifier {  
    protected final Notifier wrappedNotifier;  
  
    public NotifierDecorator(Notifier notifier) {  
        this.wrappedNotifier = notifier;  
    }  
  
    @Override  
    public void send(String message) {  
        // Делегируем вызов обёрнутому объекту  
    }  
}
```

```

        wrappedNotifier.send(message);
    }
}

```

Конкретный декоратор:

```

// Декоратор, добавляющий шифрование сообщения
public class EncryptedNotifierDecorator extends
NotifierDecorator {
    public EncryptedNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    @Override
    public void send(String message) {
        String encryptedMessage = encrypt(message);
        super.send(encryptedMessage);
    }

    private String encrypt(String message) {
        return "[ENCRYPTED] " + message;
    }
}

```

Пояснение к примеру:

- **Notifier** – интерфейс **Component**, определяющий единый контракт для отправки уведомлений.
- **EmailNotifier** – **ConcreteComponent**, базовая реализация отправки по email.
- **NotifierDecorator** – абстрактный декоратор, хранит ссылку на обёрнутый объект и делегирует вызов.
- **EncryptedNotifierDecorator** – декоратор, который модифицирует данные до передачи по цепочке (поведение добавляется перед делегированием).

Примеры использования:

- Java IO: **BufferedInputStream**, **GZIPInputStream**, **CipherInputStream** – классические примеры декораторов, оборачивающих **InputStream**.

- Логирование: оборачивание класса/репозитория/другого хранилища позволяет логировать все обращения к нему
- Валидация и трансформация данных: Цепочка декораторов может последовательно валидировать, нормализовать и обогащать данные перед сохранением.

Родственные паттерны:

1. Proxy – Имеет схожую структуру (обёртка), но решает другую задачу: Proxy контролирует доступ к объекту, а Декоратор добавляет новое поведение. Proxy обычно не изменяет интерфейс и может управлять жизненным циклом объекта, тогда как Декоратор всегда реализует тот же интерфейс и фокусируется на расширении функциональности.
2. Adapter – Изменяет интерфейс объекта для обеспечения совместимости с другим кодом. Декоратор сохраняет интерфейс, но расширяет поведение.
3. Chain of responsibility – Похож на цепочку декораторов, где каждый может обработать запрос или передать дальше. Однако в Chain of Responsibility обработчики могут прерывать цепочку, а декораторы всегда передают вызов дальше (хотя могут и не передавать, но это нарушает ожидаемое поведение).

Поведенческий паттерн «Цепочка ответственности» («Chain of respobsibility»)

Название: Цепочка ответственности (Chain of responsibility)

Тип: Поведенческий паттерн

Назначение: Для организации в системе уровней ответственности, позволяет установить, должно ли сообщение обрабатываться на том уровне, где оно было получено, или же оно должно передаваться для обработки другому объекту

Другие названия: Object chain, Pipeline, Chain of command

Мотивация (решаемые проблемы):

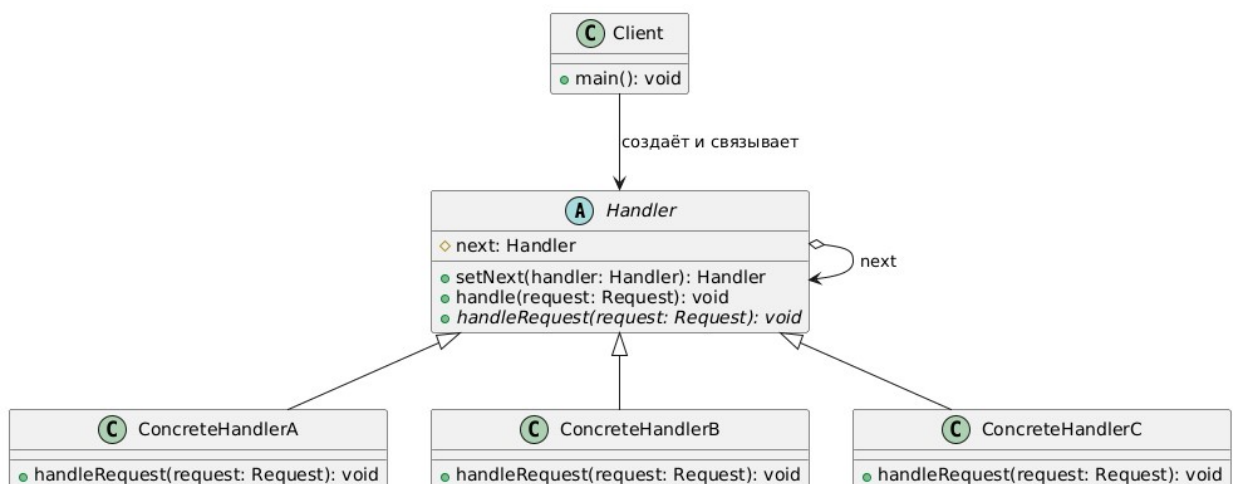
1. Жёсткая привязка отправителя к получателю – если отправитель знает, какой именно объект должен обработать запрос, это создаёт тесную связанность и затрудняет модификацию системы.
2. Необходимость условной логики выбора обработчика – без паттерна пришлось бы использовать разветвлённые конструкции if-else или switch для определения того, кто должен обработать запрос.
3. Динамическое изменение набора обработчиков – набор объектов, способных обработать запрос, может меняться во время выполнения системы (например, добавляются новые уровни логирования или этапы валидации).
4. Нарушение принципа единственной ответственности – отправитель вынужден знать не только о своей основной задаче, но и о том, как маршрутизировать запросы к обработчикам.

Условия, в которых целесообразно применять паттерн:

- Более одного объекта может обработать запрос, причём конкретный обработчик заранее неизвестен и определяется во время выполнения.

- Необходимо передать запрос одному из нескольких объектов, не указывая получателя явно.
- Набор обработчиков может меняться динамически
- Обработка запроса может быть прервана — если какой-то обработчик может полностью обработать запрос и не требует дальнейшей передачи.
- Требуется последовательная обработка запроса несколькими обработчиками — когда каждый обработчик может внести свой вклад в обработку, и все они должны быть вызваны по очереди.
- Необходимо избежать жёсткой связи между отправителем и получателем, обеспечивая гибкость и возможность лёгкого изменения маршрутизации запросов.

Структура паттерна (диаграмма классов):



1. Абстрактный обработчик — абстрактный класс Handler определяет единый интерфейс для всех обработчиков. Он хранит ссылку на следующий обработчик (next), предоставляет метод setNext() для построения цепочки и объявляет абстрактный метод handleRequest(), который реализуют конкретные обработчики.
2. Построение цепочки — метод setNext() возвращает переданный обработчик, что позволяет строить цепочку с помощью fluent-интерфейса (например, handlerA.setNext(handlerB).setNext(handlerC)).

Связь между обработчиками представлена композицией (Handler о--> Handler).

3. Конкретные обработчики – ConcreteHandlerA, ConcreteHandlerB и ConcreteHandlerC реализуют метод `handleRequest()`. Каждый из них решает, может ли он обработать запрос (обычно по типу, условию или уровню приоритета). Если может – выполняет обработку и завершает прохождение цепочки; если нет – передаёт запрос следующему обработчику через вызов `next.handleRequest()`.

Объекты и паттерны, используемые в данном паттерне:

Handler (обработчик)	Абстрактный класс или интерфейс, определяющий общий контракт для всех обработчиков в цепочке. Содержит ссылку на следующий элемент.
ConcreteHandler	Конкретная реализация обработчика. Содержит логику принятия решения – обрабатывать запрос или передавать дальше.
Client	Иницирует запрос, передавая его первому обработчику в цепочке. Не имеет прямой зависимости от конкретных обработчиков.
Request	Контейнер для данных запроса. Может содержать тип запроса, параметры, контекст выполнения.

Результаты работы паттерна:

Положительные результаты:

1. Ослабление связанности – отправитель запроса не зависит от конкретных обработчиков. Обработчики могут добавляться, удаляться или изменяться без модификации клиентского кода.
2. Гибкость распределения обязанностей – ответственность за обработку запроса может быть динамически перераспределена между объектами.
3. Принцип единственной ответственности – каждый обработчик отвечает только за свой тип запросов или этап обработки, что упрощает тестирование и поддержку.
4. Динамическая конфигурация – цепочка может быть построена и перенастроена во время выполнения в зависимости от контекста (например, роли пользователя, системной нагрузки).
5. Открытость для расширения – новые обработчики могут быть добавлены в цепочку без изменения существующего кода, что соответствует принципу открытости/закрытости.

Отрицательные результаты и ограничения:

1. Гарантия обработки не обеспечивается – запрос может дойти до конца цепочки и остаться необработанным, если ни один обработчик не взял на себя ответственность.
2. Сложность отладки – трассировка пути запроса через цепочку может быть затруднена, особенно в длинных и динамически построенных цепочках.
3. Риск заикливания – при неправильной реализации или настройке цепочка может замкнуться сама на себя, вызывая бесконечные циклы.

4. Неочевидность логики – последовательность обработчиков может быть скрыта от клиента, и понимание того, как будет обработан запрос, требует анализа всей цепочки.

Рекомендации по применению:

1. Определить чёткий критерий обработки – каждый обработчик должен иметь понятное условие, по которому он решает, может ли он обработать запрос. Это могут быть тип запроса, диапазон значений, наличие прав или состояние системы.
2. Реализовать терминатор цепочки – последний обработчик в цепочке должен либо обрабатывать все оставшиеся запросы, либо явно сигнализировать об отсутствии обработчика.
3. Документировать порядок обработчиков – если порядок влияет на результат (например, валидация перед аутентификацией), это должно быть явно зафиксировано.
4. Рассмотреть использование строителя (Builder) – для создания сложных цепочек с множеством обработчиков удобно использовать паттерн Builder, который обеспечивает читаемость и правильный порядок.
5. Быть осторожными с циклическими ссылками – при динамическом изменении цепочек убедиться, что обработчик не ссылается на себя напрямую или косвенно.

Пример кода (обработка запросов службой поддержки):

Общий обработчик:

```
public abstract class SupportHandler {
    protected SupportHandler nextHandler;

    public SupportHandler setNext(SupportHandler handler) {
        this.nextHandler = handler;
        return handler;
    }
}
```

```

public void handle(Ticket ticket) {
    if (canHandle(ticket)) {
        process(ticket);
    } else if (nextHandler != null) {
        nextHandler.handle(ticket);
    } else {
        System.out.println("Заявка #" + ticket.getId() +
Обратитесь к администратору.");
    }
}

```

```

protected abstract boolean canHandle(Ticket ticket);

```

```

protected abstract void process(Ticket ticket);

```

```

}

```

Класс запроса:

```

public class Ticket {
    private final int id;
    private final String description;
    private final Severity severity;
    private boolean resolved = false;

    public enum Severity {
        LOW, MEDIUM, HIGH, CRITICAL
    }

    public Ticket(int id, String description, Severity severity)
{
    this.id = id;
    this.description = description;
    this.severity = severity;
}

    public int getId() { return id; }
    public String getDescription() { return description; }
    public Severity getSeverity() { return severity; }
    public boolean isResolved() { return resolved; }
    public void setResolved(boolean resolved) { this.resolved =
resolved; }
}

```

```
}
```

Конкретные обработчики для каждого уровня поддержки:

```
public class Level1Support extends SupportHandler {
    @Override
    protected boolean canHandle(Ticket ticket) {
        return ticket.getSeverity() == Ticket.Severity.LOW;
    }

    @Override
    protected void process(Ticket ticket) {
        System.out.println("L1 поддержка обрабатывает заявку #"
+ ticket.getId() +
                                ": " + ticket.getDescription());
        ticket.setResolved(true);
        System.out.println("Заявка #" + ticket.getId() + "
решена на уровне L1");
    }
}
```

```
public class Level2Support extends SupportHandler {
    @Override
    protected boolean canHandle(Ticket ticket) {
        return ticket.getSeverity() == Ticket.Severity.MEDIUM;
    }

    @Override
    protected void process(Ticket ticket) {
        System.out.println("L2 поддержка обрабатывает заявку #"
+ ticket.getId() +
                                ": " + ticket.getDescription());
        ticket.setResolved(true);
        System.out.println("Заявка #" + ticket.getId() + "
решена на уровне L2");
    }
}
```

```
public class Level3Support extends SupportHandler {
    @Override
    protected boolean canHandle(Ticket ticket) {
        return ticket.getSeverity() == Ticket.Severity.HIGH;
    }
}
```



```

        @Override
        protected void process(Ticket ticket) {
            System.out.println("L3 поддержка обрабатывает заявку #"
+ ticket.getId() +
                                ": " + ticket.getDescription());
            ticket.setResolved(true);
            System.out.println("Заявка #" + ticket.getId() + "
решена на уровне L3");
        }
    }

    public class CriticalIssueHandler extends SupportHandler {
        @Override
        protected boolean canHandle(Ticket ticket) {
            return ticket.getSeverity() == Ticket.Severity.CRITICAL;
        }

        @Override
        protected void process(Ticket ticket) {
            System.out.println("!!! КРИТИЧЕСКАЯ ЗАЯВКА !!!");
            System.out.println("Заявка #" + ticket.getId() + ": " +
ticket.getDescription());
            System.out.println("Немедленно уведомляем технического
директора");
            ticket.setResolved(true);
            System.out.println("Критическая заявка #" +
ticket.getId() + " взята в экстренную работу");
        }
    }

    public class DefaultHandler extends SupportHandler {
        @Override
        protected boolean canHandle(Ticket ticket) {
            return true;
        }

        @Override
        protected void process(Ticket ticket) {
            System.out.println("Стандартная процедура для заявки #"
+ ticket.getId() +
                                ". Перенаправляем в общую очередь.");
            ticket.setResolved(true);
        }
    }

```

```
    }  
}
```

Клиентская часть:

```
public class SupportSystem {  
    public static void main(String[] args) {  
        Level1Support level1 = new Level1Support();  
        Level2Support level2 = new Level2Support();  
        Level3Support level3 = new Level3Support();  
        CriticalIssueHandler critical = new  
CriticalIssueHandler();  
        DefaultHandler defaultHandler = new DefaultHandler();  
        level1.setNext(level2)  
            .setNext(level3)  
            .setNext(critical)  
            .setNext(defaultHandler);  
  
        Ticket ticket1 = new Ticket(101, "Не могу войти в  
аккаунт", Ticket.Severity.LOW);  
        Ticket ticket2 = new Ticket(102, "Приложение зависает  
при запуске", Ticket.Severity.MEDIUM);  
        Ticket ticket3 = new Ticket(103, "База данных не  
отвечает", Ticket.Severity.HIGH);  
        Ticket ticket4 = new Ticket(104, "Система не работает у  
всех клиентов", Ticket.Severity.CRITICAL);  
        Ticket ticket5 = new Ticket(105, "Мелкая опечатка в  
интерфейсе", Ticket.Severity.LOW);  
  
        System.out.println("=== Обработка заявок ===\n");  
        System.out.println("--- Заявка #101 ---");  
        level1.handle(ticket1);  
        System.out.println("\n--- Заявка #102 ---");  
        level1.handle(ticket2);  
        System.out.println("\n--- Заявка #103 ---");  
        level1.handle(ticket3);  
        System.out.println("\n--- Заявка #104 ---");  
        level1.handle(ticket4);  
        System.out.println("\n--- Заявка #105 ---");  
        level1.handle(ticket5);  
    }  
}
```

Пояснение к примеру:

- **SupportHandler** – абстрактный обработчик, содержит ссылку на следующий элемент цепочки (**nextHandler**) и метод **handle()**, который либо обрабатывает запрос, либо передаёт дальше.
- **Ticket** – объект запроса, содержит информацию о заявке: идентификатор, описание и уровень сложности.
- Конкретные обработчики (**Level1Support**, **Level2Support**, **Level3Support**, **CriticalIssueHandler**) – каждый реализует методы **canHandle()** (проверка, может ли он обработать запрос) и **process()** (логика обработки).
- Клиент (**SupportSystem**) – строит цепочку, устанавливая связи между обработчиками с помощью метода **setNext()**, и затем передаёт запросы первому обработчику.
- Обработчик может обработать запрос и не передавать его дальше (как в примере с **Level1Support**).
- Если обработчик не может обработать запрос, он передаёт его следующему.
- Цепочка может иметь терминатор (в примере **CriticalIssueHandler** и **DefaultHandler**), который гарантирует, что любой запрос будет обработан.
- Порядок в цепочке важен: заявка уровня **LOW** будет обработана на первом уровне и не дойдёт до более высоких уровней.

Примеры использования:

- Аутентификация и авторизация: Цепочка фильтров безопасности: проверка сессии, проверка прав доступа, проверка IP-адреса, проверка лимитов запросов.
- Логирование: Цепочка логгеров: консольный логгер, файловый логгер, удалённый логгер. Каждый решает, логировать ли сообщение в зависимости от уровня важности.

- Валидация данных: Последовательная валидация: проверка обязательных полей, проверка формата, проверка бизнес-правил, проверка уникальности в БД.

Родственные паттерны:

1. Command – Часто используется вместе с Chain of Responsibility: запрос оформляется как объект Command и передаётся по цепочке. Команды могут быть поставлены в очередь, логированы или отменены.
2. Decorator – Имеет схожую структуру цепочки, но разное назначение. Decorator обязательно вызывает следующий элемент (добавляя поведение), а Chain of Responsibility может прервать передачу. Decorator фокусируется на расширении функциональности, Chain of Responsibility – на распределении обработки.
3. Pipeline – Близкий родственник, особенно в функциональном программировании. Pipeline предполагает, что каждый этап обязательно применяется к данным, в то время как Chain of Responsibility может пропускать этапы.

Порождающий паттерн «Метод фабрики» («Factory method»)

Название: Метод фабрики (Factory method)

Тип: Порождающий паттерн

Назначение: Определяет стандартный метод создания объекта, не связанный с вызовом конструктора, оставляя решение о том, какой именно объект создавать, за подклассами

Мотивация (решаемые проблемы):

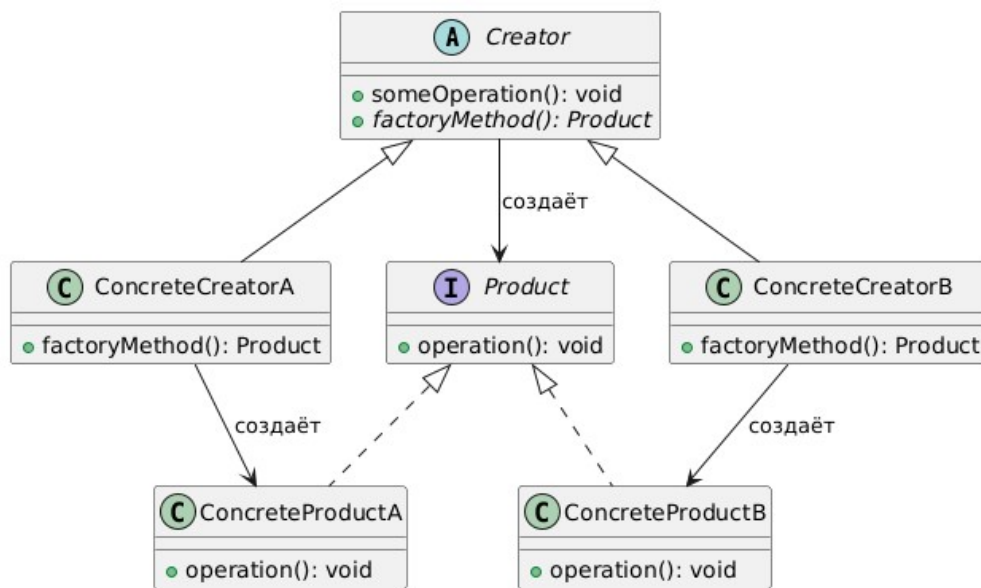
1. Жёсткая привязка к конкретным классам – использование оператора `new` напрямую связывает код с конкретными реализациями, что затрудняет замену одной реализации на другую.
2. Нарушение принципа открытости/закрытости – при добавлении новых типов продуктов приходится модифицировать существующий код, добавляя новые условные операторы.
3. Сложность тестирования – невозможно подменить реальные зависимости на тестовые двойники (моки, стабы) без изменения производственного кода.
4. Дублирование логики создания – если создание объекта требует сложной инициализации, эта логика может дублироваться в разных местах приложения.

Условия, в которых целесообразно применять паттерн:

- Код должен работать с объектами, конкретный тип которых неизвестен на этапе компиляции – тип определяется во время выполнения на основе конфигурации, входных данных или условий окружения.
- Класс не может предвидеть типы объектов, которые должны быть созданы его подклассами – создание делегируется подклассам, которые могут переопределить фабричный метод.

- Необходимо централизовать логику создания объектов – сложная инициализация, управление ресурсами или настройка параметров вынесены в одно место.
- Создание объекта требует условной логики – выбор типа объекта зависит от внешних факторов, и эту логику нужно изолировать от основной бизнес-логики.

Структура паттерна (диаграмма классов):



1. Создатель (Creator) – абстрактный класс Creator объявляет фабричный метод `factoryMethod()`, который возвращает объект типа `Product`. Также может содержать реализацию `someOperation()`, которая использует фабричный метод для работы с продуктом, что обеспечивает единую бизнес-логику при разных типах создаваемых объектов.
2. Фабричный метод – абстрактный метод `factoryMethod()` делегирует логику создания конкретного продукта подклассам. Это позволяет отделить код создания объекта от кода его использования, следуя принципу инверсии зависимостей.
3. Конкретные создатели – `ConcreteCreatorA` и `ConcreteCreatorB` переопределяют фабричный метод, возвращая соответствующие

конкретные продукты (ConcreteProductA и ConcreteProductB). Каждый создатель специализирован на создании определённого типа продукта, но клиент работает с ними через общий интерфейс Creator.

4. Абстракция продукта – интерфейс Product определяет общий контракт (operation()) для всех объектов, которые могут быть созданы фабричным методом. Это обеспечивает полиморфную работу с продуктами в методах создателя.
5. Конкретные продукты – ConcreteProductA и ConcreteProductB реализуют интерфейс Product, предоставляя конкретную реализацию бизнес-логики. Они могут быть абсолютно разными по внутреннему устройству, но единообразно использоваться через общий интерфейс.

Объекты и паттерны, используемые в данном паттерне:

Creator (создатель)	Абстрактный класс или интерфейс, содержащий фабричный метод. Может содержать бизнес-логику, использующую создаваемые продукты.
ConcreteCreator (конкретный создатель)	Переопределяет фабричный метод для создания конкретного продукта. Может иметь собственные параметры или конфигурацию.
Product (продукт)	Абстрактный тип, определяющий интерфейс создаваемых объектов.
ConcreteProduct (конкретный продукт)	Реализует интерфейс продукта. Может иметь специфические особенности, скрытые за общим интерфейсом.
Singleton	Создатель или продукт могут быть синглтонами, особенно если создание объекта требует единственного экземпляра.

Dependency Injection	Фабричный метод может использоваться как альтернатива внедрению зависимостей, особенно когда тип создаваемого объекта зависит от состояния создателя.
----------------------	---

Результаты работы паттерна:

Положительные результаты:

1. Ослабление связанности – код работает с абстрактным интерфейсом Product, не завися от конкретных реализаций. Это упрощает замену одной реализации на другую.
2. Соответствие принципу открытости/закрытости – новые типы продуктов можно добавлять, создавая новые подклассы создателя, без изменения существующего кода.
3. Централизация логики создания – сложная логика инициализации и конфигурации объектов собрана в одном месте (фабричном методе), что упрощает поддержку.
4. Упрощение тестирования – возможность подменять реальные продукты тестовыми двойниками через создание подклассов-заглушек.
5. Повторное использование кода – общая бизнес-логика, работающая с продуктами, находится в базовом классе и не дублируется в подклассах.

Отрицательные результаты и ограничения:

1. Увеличение количества классов – для каждого типа продукта требуется свой конкретный создатель, что может привести к разрастанию иерархии классов.
2. Усложнение архитектуры – появляется дополнительный уровень абстракции, который может быть избыточен для простых сценариев создания объектов.

3. Сложность наследования – клиент должен наследоваться от создателя, чтобы переопределить фабричный метод, что не всегда возможно (например, если класс уже имеет предка).
4. Проблемы с приватными конструкторами – если продукт имеет приватный конструктор, фабричный метод должен находиться внутри самого класса продукта или использовать другие механизмы создания.

Рекомендации по применению:

1. Называть фабричный метод осмысленно – имя метода должно отражать, что он создаёт (например, `createLogger()`, `buildConnection()`, `makeDocument()`).
2. Рассмотреть возможность реализации фабричного метода с параметрами – параметризованный фабричный метод может создавать разные типы продуктов на основе входных данных, но надо быть осторожным с нарушением принципа подстановки Лисков.
3. Избегать сложной логики в фабричном методе – фабричный метод должен отвечать только за создание объекта. Сложную бизнес-логику выносить в отдельные методы.
4. Документировать контракт фабричного метода – укажите, какие исключения могут быть выброшены, может ли метод вернуть `null`, и какие гарантии даёт создаваемый объект.
5. Рассмотреть альтернативы – для простых случаев создания объектов может быть достаточно статического фабричного метода или конструктора.
6. Соблюдайте принцип единственной ответственности – класс создателя не должен заниматься чем-то кроме своей основной бизнес-логики и создания продуктов.

Пример кода (обработка запросов службой поддержки):

Пример демонстрирует систему доставки, где разные типы транспорта (грузовик, корабль, самолёт) создаются через фабричный метод в зависимости от типа заказа.

Интерфейс продукта (транспорт):

```
public interface Transport {  
    void deliver(String cargo);  
    String getEstimatedTime();  
    double getCost();  
}
```

Конкретные продукты:

```
public class Truck implements Transport {  
    @Override  
    public void deliver(String cargo) {  
        System.out.println("Доставка груза '" + cargo + "' по  
суше на грузовике");  
    }  
  
    @Override  
    public String getEstimatedTime() {  
        return "2-3 дня";  
    }  
  
    @Override  
    public double getCost() {  
        return 1500.0;  
    }  
}
```

```
public class Ship implements Transport {  
    @Override  
    public void deliver(String cargo) {  
        System.out.println("Доставка груза '" + cargo + "' по  
морю на корабле");  
    }  
  
    @Override  
    public String getEstimatedTime() {
```

```

        return "7-14 дней";
    }

    @Override
    public double getCost() {
        return 800.0;
    }
}

public class Airplane implements Transport {
    @Override
    public void deliver(String cargo) {
        System.out.println("Доставка груза '" + cargo + "' по
воздуху на самолёте");
    }

    @Override
    public String getEstimatedTime() {
        return "1-2 дня";
    }

    @Override
    public double getCost() {
        return 3500.0;
    }
}

```

Абстрактный создатель:

```

public abstract class LogisticsCompany {

    protected abstract Transport createTransport();

    public void processDelivery(String cargo) {
        Transport transport = createTransport();

        System.out.println("=== Обработка заказа на доставку
===");

        System.out.println("Тип транспорта: " +
transport.getClass().getSimpleName());

        System.out.println("Стоимость доставки: " +
transport.getCost() + " руб.");

        System.out.println("Расчётное время: " +
transport.getEstimatedTime());

        transport.deliver(cargo);
    }
}

```

```

        System.out.println("Заказ выполнен!\n");
    }

    public double calculatePrice(String cargo) {
        Transport transport = createTransport();
        double baseCost = transport.getCost();
        return baseCost * 1.2;
    }
}

```

Конкретные создатели:

```

public class GroundLogistics extends LogisticsCompany {
    @Override
    protected Transport createTransport() {
        return new Truck();
    }
}

```

```

public class SeaLogistics extends LogisticsCompany {
    @Override
    protected Transport createTransport() {
        return new Ship();
    }
}

```

```

public class AirLogistics extends LogisticsCompany {
    @Override
    protected Transport createTransport() {
        return new Airplane();
    }
}

```

```

public class DynamicLogistics extends LogisticsCompany {
    private final DeliveryType deliveryType;

    public enum DeliveryType {
        EXPRESS, ECONOMY, STANDARD
    }

    public DynamicLogistics(DeliveryType type) {
        this.deliveryType = type;
    }
}

```

```

    }

    @Override
    protected Transport createTransport() {
        switch (deliveryType) {
            case EXPRESS:
                return new Airplane();
            case ECONOMY:
                return new Ship();
            case STANDARD:
            default:
                return new Truck();
        }
    }
}

```

Пояснение к примеру:

- **Transport** – интерфейс продукта, определяющий контракт для всех транспортных средств. Клиентский код (в данном случае метод `processDelivery()`) работает с объектами через этот интерфейс.
- **Truck, Ship, Airplane** – конкретные продукты, реализующие интерфейс **Transport**. Каждый имеет свою логику доставки, время и стоимость.
- **LogisticsCompany** – абстрактный создатель, объявляющий фабричный метод `createTransport()`. Содержит общую бизнес-логику `processDelivery()`, которая использует создаваемый транспорт. Обратите внимание, что `processDelivery()` не знает, какой именно транспорт будет создан – это решение делегировано подклассам.
- **GroundLogistics, SeaLogistics, AirLogistics** – конкретные создатели, переопределяющие фабричный метод для создания соответствующего типа транспорта.
- **DynamicLogistics** – демонстрирует параметризованный фабричный метод, где выбор конкретного продукта зависит от состояния создателя (типа доставки, переданного в конструктор).

- Фабричный метод `createTransport()` объявлен как `protected`, чтобы его могли переопределять подклассы, но он не предназначен для прямого вызова из клиентского кода.
- Базовый класс содержит готовую бизнес-логику (`processDelivery()`), которая полагается на фабричный метод. Это позволяет переиспользовать логику для всех типов транспорта.

Примеры использования:

- Стандартная библиотека Java: метод `iterator()` является фабричным методом, создающим итератор для конкретной коллекции.
- Парсеры: `DocumentParser` с фабричным методом `createParser()` для создания парсера PDF, DOCX, HTML в зависимости от типа файла.
- Библиотеки для работы с БД: Создание соединений с базой данных (`createConnection()`) для разных СУБД (MySQL, PostgreSQL, Oracle) через фабричный метод.

Родственные паттерны:

1. Abstract factory – Часто реализуется с помощью фабричных методов. `Abstract Factory` создаёт семейства связанных объектов, а `Factory Method` – один объект. `Abstract Factory` может содержать несколько фабричных методов.
2. Singleton – Может использоваться совместно: фабричный метод может возвращать экземпляр синглтона. Иногда фабричный метод реализуется как синглтон, если создатель должен существовать в единственном экземпляре.
3. Dependency injection – Альтернативный подход к созданию объектов. Вместо того чтобы класс сам создавал зависимости через фабричный метод, зависимости передаются извне. DI часто предпочтительнее для тестируемости и гибкости.
4. Template method – Фабричный метод часто является частью шаблонного метода.

Паттерн параллельного программирования «Блокировка чтения/записи» («Read/Write Lock»)

Название: Блокировка чтения/записи (Read/Write Lock).

Тип: Паттерн параллельного программирования

Альтернативные названия: Multiple readers/Single writer lock, Shared-exclusive lock, Concurrent Read / Exclusive Write Lock

Назначение: Обеспечивает альтернативный доступ к ресурсам, в случае, если одни операции могут совместно использовать тот или иной ресурс одновременно, а другие этого делать не могут

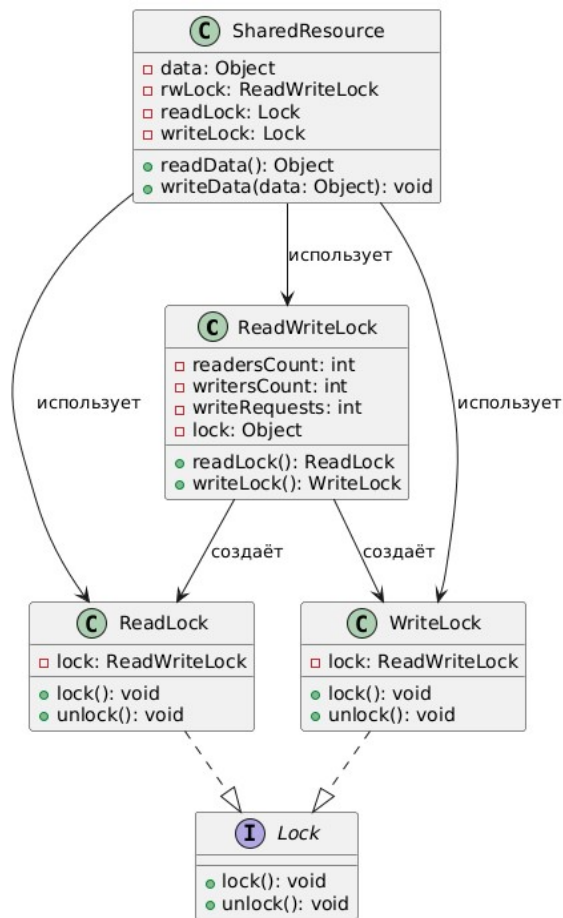
Мотивация (решаемые проблемы):

1. Избыточная блокировка при чтении – если несколько потоков только читают данные, использование обычной блокировки заставляет их выполняться последовательно, хотя параллельное чтение безопасно.
2. Снижение пропускной способности – в системах с высокой интенсивностью чтения и низкой интенсивностью записи традиционные блокировки создают узкое место, неоправданно ограничивая параллелизм.
3. Неоптимальное использование ресурсов – многопроцессорные системы могут эффективно выполнять множество читающих потоков одновременно, но стандартные механизмы синхронизации не используют это преимущество.
4. Усложнение кода при ручной оптимизации – попытки вручную реализовать раздельное управление чтением и записью часто приводят к сложным, подверженным ошибкам конструкциям.

Условия, в которых целесообразно применять паттерн:

- Разделяемый ресурс часто читается и редко изменяется – соотношение операций чтения к записи значительно, что делает оптимизацию чтения оправданной.
- Операции чтения не изменяют состояние ресурса – чтение должно быть безопасным для параллельного выполнения (нет побочных эффектов).
- Требуется максимизировать параллелизм – система должна эффективно использовать многопроцессорные/многоядерные архитектуры.
- Допустима асимметрия между читателями и писателями – можно предоставить приоритет одной из сторон в зависимости от требований системы.
- Операции чтения могут быть длительными – если чтение занимает значительное время, блокировка других читателей становится особенно неэффективной.

Структура паттерна (диаграмма классов):



1. Ядро синхронизации – класс `ReadWriteLock` управляет счётчиками читателей (`readersCount`), писателей (`writersCount`) и ожидающих запросов на запись (`writeRequests`). Он инкапсулирует всю сложную логику координации доступа, определяя, когда читатели могут получить доступ одновременно, а когда писатель должен получить эксклюзивный доступ.
2. Разделение интерфейсов – класс предоставляет методы `readLock()` и `writeLock()`, которые возвращают объекты `ReadLock` и `WriteLock` соответственно. Это реализует принцип разделения интерфейсов (ISP): клиенты получают только те методы, которые им нужны, и не могут случайно вызвать несоответствующий тип блокировки.
3. Реализация блокировок – классы `ReadLock` и `WriteLock` реализуют общий интерфейс `Lock` с методами `lock()` и `unlock()`. `ReadLock` позволяет

множественным потокам одновременно захватывать блокировку, пока нет активного писателя. WriteLock обеспечивает эксклюзивный доступ – только один поток может захватить его, и только когда нет активных читателей.

4. Защищаемый ресурс – класс SharedResource представляет собой бизнес-данные (data), которые требуют потокобезопасного доступа. Он содержит экземпляр ReadWriteLock и ссылки на объекты чтения и записи. Метод readData() захватывает read-блокировку, обеспечивая безопасное параллельное чтение. Метод writeData() захватывает write-блокировку, гарантируя, что во время записи никто не читает и не пишет.

Объекты и паттерны, используемые в данном паттерне:

ReadWriteLock	Центральный координатор доступа. Управляет очередями ожидания, счётчиками активных читателей/писателей, реализует политику приоритетов.
ReadLock	Легковесный объект, представляющий блокировку чтения. Поток получает его для выполнения операций чтения.
WriteLock	Легковесный объект, представляющий блокировку записи. Обеспечивает эксклюзивный доступ.
Condition Variables	Используются внутри для ожидания потоков, когда блокировка недоступна (например, wait()/notify() в Java или pthread_cond_wait в C).
SharedResource	Ресурс, к которому синхронизируется доступ. Обычно содержит собственно данные и методы для работы с ними.

ReadWriteLock	Центральный координатор доступа. Управляет очередями ожидания, счётчиками активных читателей/писателей, реализует политику приоритетов.
Proxy	ReadLock и WriteLock выступают как прокси к реальному механизму блокировки, скрывая сложность управления счётчиками.
Strategy	Различные стратегии предоставления доступа (приоритет читателям, приоритет писателям, честная очередь) могут быть реализованы как взаимозаменяемые стратегии.

Результаты работы паттерна:

Положительные результаты:

1. Повышение пропускной способности – в системах с преобладанием операций чтения параллелизм возрастает многократно, так как читатели не блокируют друг друга.
2. Эффективное использование многопроцессорных систем – несколько ядер могут одновременно выполнять операции чтения, что улучшает масштабируемость.
3. Снижение задержек – читатели не ждут завершения других читателей, что уменьшает время ожидания для операций чтения.
4. Гибкость в выборе политики – можно настраивать приоритеты (читатели имеют преимущество, писатели имеют преимущество, честная очередь) в зависимости от требований системы.
5. Предотвращение состояний гонки – гарантирует, что ни один писатель не получит доступ, пока есть активные читатели, и наоборот.

6. Улучшение отзывчивости – приложения с интенсивным чтением (например, веб-серверы, кэши) становятся более отзывчивыми под нагрузкой.

Отрицательные результаты и ограничения:

1. Сложность реализации – корректная реализация требует тщательного управления счётчиками и условиями ожидания, особенно с учётом повторного входа (reentrancy).
2. Риск «голодания» – при приоритете читателей писатели могут никогда не получить доступ, если поток читателей непрерывен. При приоритете писателей читатели могут голодать.
3. Накладные расходы – даже при отсутствии конкуренции, операции блокировки/разблокировки дороже, чем у простых взаимных исключений, из-за необходимости проверки дополнительных счётчиков.
4. Неэффективность при частых записях – если операции записи происходят часто, выгода от параллельного чтения нивелируется, а накладные расходы остаются.

Рекомендации по применению:

1. Оценить соотношение чтения/записи – применять паттерн только если чтение значительно преобладает над записью (рекомендуется соотношение не менее 10:1). Для сбалансированной нагрузки или преобладания записи использовать обычные блокировки.
2. Выбрать подходящую политику приоритетов.
3. Быть осторожными с эскалацией блокировок (lock escalation) – не пытаться в одном потоке сначала получить read lock, а затем и write lock. Это может привести к deadlock.

4. Минимизировать время удержания блокировок – выполняйте операции чтения и записи как можно быстрее. Особенно критично для write lock, который блокирует всех читателей и других писателей.
5. Не использовать read lock как защиту от изменений на длительное время.
6. Тестировать под нагрузкой – проблемы с голоданием или дедлоками часто проявляются только при высоком уровне конкурентности.

Пример кода (реализация в стандартной библиотеке Java):

Кэш с разделяемым доступом для чтения и эксклюзивным для записи:

```
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class DataCache<K, V> {
    private final Map<K, V> storage = new HashMap<>();
    private final ReentrantReadWriteLock rwLock = new
ReentrantReadWriteLock();
    private final ReentrantReadWriteLock.ReadLock readLock =
rwLock.readLock();
    private final ReentrantReadWriteLock.WriteLock writeLock =
rwLock.writeLock();

    // Чтение – множественный доступ
    public V get(K key) {
        readLock.lock();
        try {
            return storage.get(key);
        } finally {
            readLock.unlock();
        }
    }

    // Запись – эксклюзивный доступ
    public void put(K key, V value) {
        writeLock.lock();
        try {
            storage.put(key, value);
        } finally {
            writeLock.unlock();
        }
    }
}
```

```

        writeLock.unlock();
    }
}

// Удаление - тоже эксклюзивный доступ
public V remove(K key) {
    writeLock.lock();
    try {
        return storage.remove(key);
    } finally {
        writeLock.unlock();
    }
}
}

```

Демонстрация использования:

```

public class Demo {
    public static void main(String[] args) throws
    InterruptedException {
        DataCache<String, String> cache = new DataCache<>();
        cache.put("key", "initial");

        // Несколько потоков для чтения - выполняются
        параллельно
        for (int i = 0; i < 5; i++) {
            new Thread(() -> {
                for (int j = 0; j < 3; j++) {

System.out.println(Thread.currentThread().getName() +
                                                             " читает: " +
cache.get("key"));
                }
            }).start();
        }

        // Поток для записи - выполняется, когда нет активных
        чтений
        Thread writer = new Thread(() -> {
            System.out.println("Писатель начинает запись...");
            cache.put("key", "new_value");
            System.out.println("Писатель завершил запись");
        });
    }
}

```

```
        Thread.sleep(100); // Даём читателям начать работу
        writer.start();
        writer.join();

        System.out.println("Итоговое значение: " +
            cache.get("key"));
    }
}
```

Пояснение к примеру:

- `ReentrantReadWriteLock` – Центральный механизм, управляющий отдельным доступом для чтения и записи.
- `readLock` – Блокировка чтения. Её могут одновременно удерживать множество потоков.
- `writeLock` – Блокировка записи. Может быть удержана только одним потоком и только когда нет активных `readLock`.
- Множественное чтение – метод `get()` захватывает `readLock`. Пока поток только читает, другие читатели могут работать параллельно.
- Эксклюзивная запись – метод `put()` и `remove()` захватывают `writeLock`. Пока идёт запись, все читатели и другие писатели ждут.
- Блок `try-finally` – обязательная конструкция. Блокировка всегда освобождается в `finally`, чтобы гарантировать корректность даже при исключениях.
- Важно: если поток сначала захватил `readLock`, а затем пытается захватить `writeLock` (например, для обновления значения), возникает `deadlock`. Для таких сценариев нужно сначала освободить `readLock`.

Примеры использования:

- Кэширующие системы: Кэш в памяти (например, для хранения конфигураций, результатов запросов) с высокой интенсивностью чтения и редкими обновлениями.

- Базы данных: Внутренние механизмы СУБД, где множественные транзакции могут читать данные параллельно, но запись требует эксклюзивного доступа.
- Графические интерфейсы: Доступ к моделям данных в многопоточных UI-приложениях, где рендеринг читает данные, а пользовательские действия их изменяют.
- **Родственные паттерны:**
 1. Mutex – Простейшая форма синхронизации, не различающая чтение и запись. Read/Write Lock является специализацией Mutex, предоставляющей более высокий параллелизм для операций чтения.
 2. Semaphore – Может использоваться для реализации Read/Write Lock (например, счётчик для читателей и двоичный семафор для писателей). Однако прямая реализация сложнее, чем использование специализированных средств.
 3. Thread-safe Collection – Коллекции, такие как ConcurrentHashMap, реализуют внутренние механизмы синхронизации, которые могут использовать идеи Read/Write Lock, но с более сложной внутренней оптимизацией.
 4. Monitor – Read/Write Lock часто реализуется как монитор с условными переменными. Методы lock()/unlock() синхронизированы, а ожидание реализовано через wait()/notifyAll().